

Problem A. Stellar Year

The current year is stellar, meaning all stars appear. Each star with period a_i appears only in years that are multiples of a_i . Therefore, after X years, star i will be visible if and only if $X \bmod a_i = 0$.

For each star i ($1 \leq i \leq N$), check if $X \bmod a_i == 0$. If so, include i in the answer. Then, print the total number of visible stars and their respective indices.

Complexity: $O(N)$.

Listing 1: A.cpp

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 signed main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8     int n, x;
9     cin >> n >> x;
10    vector<int> ans;
11    for (int i = 0; i < n; i++) {
12        int y;
13        cin >> y;
14        if (x % y == 0) {
15            ans.emplace_back(i + 1);
16        }
17    }
18    cout << ans.size() << endl;
19    for (auto u : ans) {
20        cout << u << ' ';
21    }
22    cout << endl;
23 }
```

Problem B. Bit Tennis 2

This is a Nim-like problem. We will prove that the Grundy number of a pile with X stones is $X \bmod 3$.

In each move, a player can remove any number of stones equal to a power of two, i.e., 2^i stones, as long as $2^i \leq X$.

Observe the behavior of powers of two modulo 3:

$$2^0 \bmod 3 = 1$$

$$2^1 \bmod 3 = 2$$

$$2^2 \bmod 3 = 1$$

$$2^3 \bmod 3 = 2$$

$$\vdots$$

That is, powers of two modulo 3 alternate between 1 and 2.

Now, suppose the current pile satisfies $X \bmod 3 = k$, where $k \in \{0, 1, 2\}$.

After removing 2^i stones, the new pile becomes:

$$X' = X - 2^i$$

Therefore:

$$X' \bmod 3 = (X \bmod 3 - 2^i \bmod 3) \bmod 3$$

Given that $2^i \bmod 3 \in \{1, 2\}$, we cannot remain in the same modulo class k after any move. Specifically:

- If $X \bmod 3 = 0$, reachable modulus are 1 and 2.
- If $X \bmod 3 = 1$, reachable modulus are 0 and 2.
- If $X \bmod 3 = 2$, reachable modulus are 0 and 1.

Thus, from state k , it is impossible to reach k again, but all other states are reachable. Therefore, the set of reachable Grundy numbers is $\{0, 1, 2\} \setminus \{k\}$, and:

$$G(X) = \text{mex}(\{0, 1, 2\} \setminus \{k\}) = k$$

This concludes the proof.

With no operations ($X = 0$), the victory follows the standard Nim analysis: Let S be the xor-sum of all initial Grundys, if $S = 0$, the starter (Julia) loses and Giovana wins; otherwise, Julia wins.

When $X > 0$, Giovana must double exactly X times before the game. Doubling a pile with value a changes its Grundy:

- If $a \bmod 3 = 0$, it doubles and remains 0.
- If $a \bmod 3 = 1$, it doubles and becomes 2.
- If $a \bmod 3 = 2$, it doubles and becomes 1.

There are two cases:

- If $S = 0$: Giovana is already winning and does not want to change the xor. It is enough if she has some pile with Grundy equal to 0 to "discard" the moves, or if X is even, so she can discard the moves in a single element without changing the xor-sum. If neither of these conditions holds, she will end up changing the xor and losing.
- If $S \neq 0$: Giovana needs to make $S = 0$, so the frequency of piles with Grundy 2 and Grundy 1 must be odd. Otherwise, one would be odd and the other even, and each doubling of a pile Giovana performs alternates which Grundy has odd frequency, preventing both from becoming even. Since both frequencies are odd, Giovana needs to apply the doublings an odd number of times; that is, either X is odd, or X can be even but she needs a pile with Grundy 0 to discard one move.

Complexity: $O(N)$.

Listing 2: B.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
```

```

4 signed main() {
5     cin.tie(0)->sync_with_stdio(false);
6
7     int n, x;
8     cin >> n >> x;
9
10    vector<int> v(n, 0);
11    for (int &u : v) cin >> u;
12
13    vector<int> freq(3, 0);
14
15    int s = 0;
16    for (auto u : v) {
17        s ^= u % 3;
18        freq[u % 3]++;
19    }
20
21    int win;
22
23    if (x) {
24        if (s == 0) {
25            win = freq[0] > 0 || x % 2 == 0;
26        } else {
27            win = freq[1] % 2 == 1 && freq[2] % 2 == 1 && (x % 2 == 1 || freq[0]
28                > 0);
29        }
30    } else {
31        win = s == 0;
32    }
33
34    cout << (win ? "Giovana" : "Julia") << endl;
35 }

```

Problem C. Song

Lívia considers the melody beautiful if, for each note (except the first), the preceding note forms an allowed pair with it.

To solve the problem, we store the N allowed pairs in a 26×26 matrix, where each entry indicates whether note u can be followed by note v . Then, we iterate over the melody and, for each adjacent pair of notes, we check in $O(1)$ whether the pair is allowed.

If all pairs are valid, the answer is "YES"; otherwise, "NO".

Complexity: $O(|S| + N)$.

Listing 3: C.cpp

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3
4 using namespace std;

```

```

5
6 signed main() {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9     string s;
10    cin >> s;
11    int n;
12    cin >> n;
13    vector<vector<int>> f(26, vector<int>(26));
14    while (n--) {
15        char v, u;
16        cin >> v >> u;
17        f[v - 'a'][u - 'a'] = 1;
18    }
19    bool ok = true;
20    for (int i = 0; i < int(s.size() - 1); i++) {
21        ok &= f[s[i] - 'a'][s[i + 1] - 'a'];
22    }
23    cout << (ok ? "SIM" : "NAO") << endl;
24 }

```

Problem D. Course Deviation

The spaceship descends 1 km per second and moves forward 1 km per second. If the descent starts at height H , when passing the i -th mountain (with i starting at 0), the spaceship will be at $H - (i + 1)$ kilometers. To avoid collisions, it is necessary that $H - (i + 1)$ is strictly greater than the height $h[i]$ of the mountain, that is, $H > h[i] + i + 1$ for all i .

Therefore, the minimum initial height that guarantees safety is $H = \max\{h[i] + i + 1\} + 1$, equivalently, $H = \max\{h[i] + i + 2\}$.

Complexity: $O(N)$.

Another solution is to perform a binary search on the initial height. For each tested height, we can simulate the process in $O(N)$.

Complexity: $O(N \times \log(\max(a_i)))$.

Listing 4: D.cpp

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4
5 int32_t main() {
6     cin.tie(0) -> sync_with_stdio(0);
7     int n;
8     cin >> n;
9     vector<int> a(n);
10    for (int &i : a) cin >> i;
11    int res = 0;
12    for (int i = 0; i < n; i++) {
13        res = max(res, a[i] + i + 2);

```

```

14     }
15     cout << res << endl;
16 }

```

Problem E. El Café

Maintain a stack that stores the accumulated gcd (greatest common divisor) of the available ingredients. Initially, for the N ingredients, build a vector *pref* with the first element equal to 0, and for each ingredient with quantity c , add $\text{gcd}(\text{pref.back()}, c)$ to the stack.

For the requests:

- Type 1: Upon receiving a new ingredient with quantity G , push $\text{gcd}(\text{pref.back()}, G)$ onto the stack.
- Type 2: Remove the K most recent ingredients from the stack. This can be done inside a while loop, as the total number of additions to the stack is $O(N + Q)$ throughout the entire execution, so the number of removals will also be within this bound.
- Type 3: For a request of X "vainillas heladas", check if the current gcd (the last element of the stack) is divisible by X , i.e., if $\text{pref.back()} \bmod X == 0$. If so, the request can be fulfilled; otherwise, it cannot.

Complexity: $O(N + Q)$.

Listing 5: E.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4
5  int main() {
6      ios_base::sync_with_stdio(0);
7      cin.tie(0);
8
9      int n, q;
10     cin >> n >> q;
11     vector<int> pref = {0};
12     for (int i = 0; i < n; i++) {
13         int c;
14         cin >> c;
15         pref.emplace_back(gcd(pref.back(), c));
16     }
17
18     while (q--) {
19         int id;
20         cin >> id;
21         if (id == 1) {
22             int c;
23             cin >> c;
24             pref.emplace_back(gcd(pref.back(), c));
25         } else if (id == 2) {
26             int k;

```

```

27         cin >> k;
28         while (k--) {
29             assert((int)pref.size() > 0);
30             pref.pop_back();
31             assert((int)pref.size() > 0);
32         }
33     } else {
34         int c;
35         cin >> c;
36         assert(c > 0);
37         cout << (pref.back() % c == 0 ? "SIM" : "NAO") << endl;
38     }
39 }
40 }

```

Problem F. Frogs or Toads?

Define $dp[i]$ as the maximum energy Eric can have after completing level i . Initially, $dp[0] = 0$. For each level i , there are two options:

- If Eric does not use the wormhole, he defeats the s_i mutants and collects e_i energy, advancing to level $i + 1$. Update: $dp[i + 1] = \max(dp[i + 1], dp[i] + (e_i - s_i))$.
- If Eric uses the wormhole, he jumps to level $\min(n, i + k)$, losing X energy. Update: $dp[\min(n, i + k)] = \max(dp[\min(n, i + k)], dp[i] - X)$.

The answer is $\max(dp[n], 0)$.

Complexity: $O(N)$.

Listing 6: F.cpp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  using ll = long long;
5
6  int32_t main() {
7      cin.tie(0) -> sync_with_stdio(0);
8
9      int n, k, x;
10     cin >> n >> k >> x;
11
12     vector<ll> s(n), e(n);
13
14     for (int i = 0; i < n; i++) {
15         cin >> s[i] >> e[i];
16     }
17
18     vector<ll> dp(n + 1, LLONG_MIN);
19     dp[0] = 0;
20     for (int i = 0; i < n; i++) {

```

```

21     dp[i + 1] = max(dp[i + 1], dp[i] + (e[i] - s[i]));
22     dp[min(n, i + k)] = max(dp[min(n, i + k)], dp[i] - x);
23 }
24
25 cout << max(dp[n], 0ll) << endl;
26 }

```

Problem G. Gargantua

Since time on Miller passes X times slower than on Earth, if Ema stayed Y years on Miller, then $X \times Y$ years have passed on Earth. Therefore, if Leo was A years old before the trip, upon Ema's return, Leo will be $A + X \times Y$ years old.

Complexity: $O(1)$.

Listing 7: G.cpp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4
5  signed main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8     int a, x, y;
9     cin >> a >> x >> y;
10    cout << a + x * y << endl;
11 }

```

Problem H. Higgs

We can perform a binary search to determine the smallest Higgs level that Risa needs to falsify. For a test with level x , we execute a 0-1 BFS on the graph (planets and connections) starting from planet 1 to planet N , where each connection with level greater than x requires a bribe (cost 1) and each connection with level less than or equal to x has cost 0. Let $d[v]$ be the minimum number of bribes needed to reach planet v . If $d[N] \leq K$, then it is possible to complete the journey with level x . Therefore, the answer is the smallest x that satisfies this condition.

Complexity: $O((N + M) \times \log(10^9))$.

Listing 8: H.cpp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  using ll = long long;
5  using ii = pair<int, int>;
6
7  const int maxn = 3e5 + 5;
8  vector<ii> adj[maxn];

```

```

9  int d[maxn];
10
11  signed main() {
12      ios_base::sync_with_stdio(false);
13      cin.tie(nullptr);
14      int n, m, k;
15      cin >> n >> m >> k;
16      for (int i = 0; i < m; i++) {
17          int v, u, w;
18          cin >> v >> u >> w;
19          --v;
20          --u;
21          adj[v].emplace_back(u, w);
22          adj[u].emplace_back(v, w);
23      }
24      auto check = [&](int x) {
25          deque<ii> q;
26          for (int i = 0; i < n; i++) {
27              d[i] = 1e9;
28          }
29          d[0] = 0;
30          q.emplace_back(0, 0);
31          while (q.size()) {
32              auto [v, dis] = q.front();
33              q.pop_front();
34              if (dis > d[v]) {
35                  continue;
36              }
37              d[v] = dis;
38              for (auto [u, w] : adj[v]) {
39                  w = w > x;
40                  if (d[v] + w < d[u]) {
41                      d[u] = d[v] + w;
42                      if (w == 1) {
43                          q.emplace_back(u, d[u]);
44                      } else {
45                          q.emplace_front(u, d[u]);
46                      }
47                  }
48              }
49          }
50          return d[n - 1] <= k;
51      };
52      int l = -1, r = 1e9 + 1;
53      while (r - l > 1) {
54          int mid = (l + r) >> 1;
55          if (check(mid))
56              r = mid;
57          else
58              l = mid;
59      }

```



```

60     cout << r << endl;
61 }

```

Problem I. Itwise Bor

To maximize the total sum, we want to avoid losing any bits from any number when performing the OR operation. We can use a greedy strategy:

- Start a group with the first element (denote it as y) and add its value to the total sum. Let now be the OR of the current group's elements (initially y) and k be the number of groups (initially 1).

Then, for each subsequent element x :

- Add x to the total sum.
- If $x \& now \neq 0$, it is not possible to include x in the current group without losing some bit; thus, restart the group by setting $now = x$ and increment k .
- Otherwise, update $now = now | x$.

The answer is the total sum of brightness values and the number of groups, which is k .

Complexity: $O(N)$.

Listing 9: I.cpp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using namespace std;
5  using ll = long long;
6
7  signed main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(nullptr);
10     int n;
11     cin >> n;
12     ll sum = 0;
13     int k = 0;
14     int now = 0;
15     for (int i = 0; i < n; i++) {
16         int x;
17         cin >> x;
18         sum += x;
19         if (x & now) {
20             now = x;
21             k++;
22         } else {
23             now |= x;
24         }
25     }
26     cout << sum << ' ' << k + 1 << endl;
27 }

```

Problem J. Jugando Fuerte

We can construct a string by concatenating the players' decks in the given order, duplicating it to simplify the implementation of the circular logic. This way, for each position in the string, we can determine which player it belongs to. For each player, we also define the range of their deck, meaning how far it extends to the left, including the decks of all players to their left, which form part of their hand.

Next, we use the Aho-Corasick algorithm to identify all occurrences of the wildcard patterns in the constructed string. For each occurrence of a pattern t_j with value X_j at position i , we can check whether the occurrence ends within the original deck of the player corresponding to position i . If it does, the player's score is updated to the maximum value among the wildcards found in their hand.

Complexity: $O(N \times \sqrt{\sum t_i})$

Note: The $\sqrt{\sum t_i}$ term arises from the following observation:

For each position i in the complete string, if there are j occurrences of distinct patterns at this position, in the worst case these j matchings may have lengths $1, 2, \dots, j$. We know that the sum of these lengths is $O(j^2)$, meaning that at any position i , we can have at most $\sqrt{\sum t_i}$ occurrences of distinct patterns.

Listing 10: J.cpp

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3 using namespace std;
4 using ll = long long;
5
6 const int N = 3e5 + 5;
7 string h[N], a[N];
8 int g[N], val[N], n, m, player[N], back[N], len = 0, fscore[N];
9 ll psum[N];
10
11 struct AC {
12     const int K = 26;
13     const char norm = 'a';
14
15     inline int get(int c) { return c - norm; }
16
17     vector<int> link, out_link, par;
18     vector<char> pch;
19     vector<vector<int>> next;
20     vector<bool> out;
21     vector<pair<int, int>> best;
22
23     AC() { node(); }
24
25     int node(int p = -1, char c = -1) {
26         link.emplace_back(-1);
27         out_link.emplace_back(-1);
28         par.emplace_back(p);
29         pch.emplace_back(c);
30         next.emplace_back(K, -1);
31         best.emplace_back(-1, -1);
```

```

32         out.emplace_back(0);
33         return (int)link.size() - 1;
34     }
35
36     void insert(const string &s, const int &value) {
37         int u = 0;
38         for (int i = 0; i < (int)s.size(); i++) {
39             auto v = next[u][get(s[i])];
40             if (v == -1) {
41                 next[u][get(s[i])] = v = node(u, s[i]);
42             }
43             u = v;
44         }
45         out[u] = true;
46         best[u] = max(best[u], {value, (int)s.size()});
47     }
48
49     int get_link(int u) {
50         if (link[u] == -1) {
51             link[u] = par[u] > 0 ? go(get_link(par[u]), pch[u]) : 0;
52         }
53         return link[u];
54     }
55
56     int go(int u, char c) {
57         if (next[u][get(c)] == -1) {
58             next[u][get(c)] = u ? go(get_link(u), c) : 0;
59         }
60         return next[u][get(c)];
61     }
62
63     int exit(int u) {
64         if (out_link[u] == -1) {
65             int v = get_link(u);
66             out_link[u] = (out[v] || !v) ? v : exit(v);
67         }
68         return out_link[u];
69     }
70
71     bool matched(int u) { return out[u] || exit(u) != 0; }
72
73 } aho;
74
75 int32_t main() {
76     cin.tie(0)->sync_with_stdio(0);
77     cin >> n;
78     for (int i = 0; i < n; i++) {
79         cin >> h[i] >> g[i];
80         len += (int)h[i].size();
81     }
82     cin >> m;

```

```

83     for (int i = 0; i < m; i++) {
84         cin >> a[i] >> val[i];
85     }
86
87     psum[0] = 0;
88     for (int i = 1; i <= 2 * n; i++) {
89         psum[i] = psum[i - 1] + (int)h[(i - 1) % n].size();
90     }
91
92     int ptr = 0;
93     for (int i = 0; i < n; i++) {
94         int qt = (int)h[i].size();
95         for (int j = 0; j < qt; j++) {
96             back[ptr] = j + 1;
97             player[ptr] = i;
98             ptr++;
99         }
100     }
101
102     for (int i = 0; i < len; i++) {
103         int p = player[i];
104         int pg = g[p];
105         back[i] += (int)(psum[p + n] - psum[p + n - pg]);
106     }
107
108     for (int i = 0; i < m; i++) {
109         aho.insert(a[i], val[i]);
110     }
111
112     string full = "";
113     for (int i = 0; i < n; i++) {
114         full += h[i];
115     }
116
117     int u = 0;
118     for (int i = 0; i < len; i++) {
119         u = aho.go(u, full[i]);
120     }
121
122     for (int i = 0; i < len; i++) {
123         u = aho.go(u, full[i]);
124         int v = u;
125         while (v > 0) {
126             if (aho.best[v].first != -1) {
127                 auto [value, sz] = aho.best[v];
128                 int j = i - sz + 1;
129                 if (back[i] >= sz) {
130                     fscore[player[i]] = max(fscore[player[i]], value);
131                 }
132             }
133             v = aho.exit(v);

```

```

134     }
135 }
136
137 for (int i = 0; i < n; i++) {
138     cout << fscore[i] << " ";
139 }
140 cout << endl;
141 }

```

Problem K. Kosmos

It can be observed that the sequence defined by:

$$F(0) = 1 \quad , \quad F(1) = 2$$

$$F(n) = F(n-1) \times F(n-2) \quad \text{for } n \geq 2$$

satisfies the property that $F(n) = 2^{f(n)}$, where:

$$f(0) = 0 \quad , \quad f(1) = 1$$

$$f(n) = f(n-1) + f(n-2) \quad \text{for } n \geq 2$$

In other words, $f(n)$ is the n -th Fibonacci number (with $f(0) = 0$ and $f(1) = 1$).

We need to compute the result modulo 998244353. For simplicity, denote this number as M , i.e., $M = 998244353$.

Due to the magnitude of $\text{fib}(n)$, we use the property of modular exponentiation, considering that:

$$2^{\text{fib}(n)} \bmod M = 2^{\text{fib}(n) \bmod \phi(M)} \bmod M$$

where $\phi(M) = M - 1$ (since M is prime). Thus, the first step is to compute $\text{fib}(n) \bmod (M - 1)$ using matrix exponentiation or another linear recurrence technique in $O(\log n)$.

Finally, we calculate $2^{\text{fib}(n) \bmod 998244352} \bmod 998244353$.

Complexity: $O(\log N)$.

Listing 11: K.cpp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  using ll = long long;
5
6  template <int mod = 998244353>
7  struct mint {
8      int val;
9
10     mint(ll v = 0) {
11         val = v % mod;
12         if (val < 0) val += mod;
13     }

```

```

14
15     mint pwr(mint b, ll e) {
16         mint res;
17         for (res = 1; e; e >>= 1, b = b * b)
18             if (e & 1) res = res * b;
19         return res;
20     }
21
22     bool operator==(mint o) { return val == o.val; }
23
24     friend mint operator*(mint a, mint o) { return (ll)a.val * o.val; }
25
26     friend mint operator+(mint a, mint o) {
27         a.val += o.val;
28         if (a.val >= mod) a.val -= mod;
29         return a;
30     }
31
32     friend mint operator-(mint a, mint o) {
33         a.val -= o.val;
34         if (a.val < 0) a.val += mod;
35         return a;
36     }
37
38     friend mint operator^(mint a, ll o) { return a.pwr(a, o); }
39
40     friend mint operator/(mint a, mint o) { return a * (o ^ (mod - 2)); }
41 };
42
43 using mint1 = mint<998244353>;
44 using mint2 = mint<998244352>;
45 using mat = vector<vector<mint2>>>;
46
47 mat mul(mat a, mat b) {
48     int n = a.size();
49     mat res(n, vector<mint2>(n, 0));
50     for (int i = 0; i < n; i++) {
51         for (int j = 0; j < n; j++) {
52             for (int k = 0; k < n; k++) {
53                 res[i][j] = res[i][j] + a[i][k] * b[k][j];
54             }
55         }
56     }
57     return res;
58 }
59
60 mat bx(mat b, ll e) {
61     int n = b.size();
62     mat res(n, vector<mint2>(n, 0));
63     for (int i = 0; i < n; i++) res[i][i] = 1;
64

```

```

65     for (; e; e >= 1) {
66         if (e & 1) res = mul(res, b);
67         b = mul(b, b);
68     }
69
70     return res;
71 }
72
73 signed main() {
74     cin.tie(00)->sync_with_stdio(true and false);
75
76     ll n;
77     cin >> n;
78
79     mat m = {{1, 1}, {1, 0}};
80
81     mat p = bx(m, n);
82
83     mint2 f = p[1][0];
84
85     mint1 res;
86
87     res = res.pwr(2, f.val);
88     cout << res.val << endl;
89 }

```

Problem L. Lango Mocos

We can model the problem as a graph, where each type of lango moco is represented by a vertex and each pair (u, v) indicates an edge between vertices u and v . The condition that two toxic lango mocos cannot be in the same bag is equivalent to partitioning the vertices into two independent sets.

Therefore, the task is possible if and only if every connected component of the graph is bipartite. To check this, we can perform a depth-first search (DFS) to color the graph with two alternating colors. If at any point a contradiction is detected (a vertex is already colored and receives an incompatible color), the separation is impossible, and the answer is "IMPOSSIBLE". Otherwise, the separation is possible, and the answer is "POSSIBLE", followed by the listing of the vertices (lango moco types) of each set.

Complexity: $O(N + M)$.

Listing 12: L.cpp

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using namespace std;
5  using ll = long long;
6
7  signed main() {
8     ios_base::sync_with_stdio(false);
9     cin.tie(nullptr);

```

```

10     int n, m;
11     cin >> n >> m;
12     vector<vector<int>> adj(n);
13     for (int i = 0; i < m; i++) {
14         int v, u;
15         cin >> v >> u;
16         --v;
17         --u;
18         adj[v].emplace_back(u);
19         adj[u].emplace_back(v);
20     }
21     vector<int> color(n, -1);
22     auto dfs = [&](auto&& self, int v, int c = 0) -> bool {
23         if (color[v] != -1) {
24             return color[v] == c;
25         }
26         color[v] = c;
27         for (auto u : adj[v]) {
28             if (!self(self, u, c ^ 1)) {
29                 return false;
30             }
31         }
32         return true;
33     };
34     for (int i = 0; i < n; i++)
35         if (color[i] == -1) {
36             if (!dfs(dfs, i, 0)) {
37                 cout << "IMPOSSIVEL" << endl;
38                 return 0;
39             }
40         }
41     cout << "POSSIVEL" << endl;
42     int z = count(color.begin(), color.end(), 0);
43     cout << z << ' ' << n - z << endl;
44     for (int j = 0; j < 2; j++) {
45         for (int i = 0; i < n; i++)
46             if (color[i] == j) {
47                 cout << i + 1 << ' ';
48             }
49         cout << endl;
50     }
51 }

```

Problem M. Giant Worms

In this problem, the multiverse is modeled as a rooted tree where each node represents a universe, and each edge (from u to v) indicates the existence of a giant wormhole allowing travel from u to v . According to the rules, if there is an edge from u to v , then universe u contains strictly more stars than universe

v . Thus, for any given set of universes, the universe "with the fewest stars that allows traveling to all of them" is, in fact, the least common ancestor (LCA) of those universes.

For each query, we are given K universes $a_1, a_2, \dots, a_K$, and the goal is to compute the sum, over all contiguous subintervals of this sequence, of $f(i, j)$, where $f(i, j)$ is the LCA of $a_i, a_{i+1}, \dots, a_j$.

A naive approach that evaluates all subintervals would have a complexity of $O(K^2)$ — impractical for large K . Therefore, we use a technique known as the **virtual tree**. The idea is as follows:

- We construct the virtual tree using the set of nodes in the query. To do this, we sort these nodes according to their entry time (obtained via DFS) in the original tree. Then, we insert the LCAs of each consecutive pair of nodes in this order, removing duplicates. The result is the smallest subgraph of the original tree that contains all the nodes of the query and is closed under the LCA operation.

Once the virtual tree is built, we process the query using the **Small to Large** technique. Our goal is to count the contribution of each node to the final answer, i.e., for each node v , we compute:

$$\text{ans}[v] = \text{the number of subarrays where } v \text{ is the LCA.}$$

To achieve this, we first solve an easier version of the problem: let $\text{acc}[v]$ be the number of subarrays such that the LCA is in the subtree of v (or v itself). Then, we obtain:

$$\text{ans}[v] = \text{acc}[v] - \sum_{u \in \{u \mid u \text{ is a child of } v\}} \text{acc}[u].$$

This equation can be interpreted as follows: the number of subarrays where v is the LCA is equal to the number of subarrays where either v or some node in the subtree of v is the LCA, minus the number of subarrays where a child of v (or some node in the subtree of a child of v) is the LCA.

To compute $\text{acc}[v]$, when reaching v in our **Small to Large** DFS, we traverse its subtree. When we reach a child u of v , we count how many subarrays within the subtree of v "pass through" u . The details of this implementation can be seen in the code below.

The final answer for the query is computed as:

$$\sum_{i=1}^K \text{ans}[a[i]] \cdot a[i]$$

The overall complexity is $O(K \log K)$ per query. Since the sum of all values of K across all queries is limited to 3×10^5 , this solution is efficient within the problem's constraints.

Listing 13: M.cpp

```

1 #include <bits/stdc++.h>
2 #define endl '\n'
3
4 using namespace std;
5 using ll = long long;
6
7 const int maxn = 1e5 + 5;
8 const int lg = 19;
9 vector<int> adj[maxn];
10 vector<int> f[maxn];
11 int up[maxn][lg], tin[maxn], tout[maxn], id[maxn], T;
12 const int maxs = maxn * lg;

```

```

13 int grupo[maxs], L[maxs], R[maxs];
14 int mask[maxn], sz[maxn];
15 ll ans[maxn], acc[maxn];
16
17 const int INF = int(2e9);
18
19 void dfs(int v, int p = 1) {
20     tin[v] = T++;
21     up[v][0] = p;
22     for (int i = 1; i < lg; i++) {
23         int u = up[v][i - 1];
24         up[v][i] = up[u][i - 1];
25     }
26     for (auto u : adj[v]) {
27         dfs(u, v);
28     }
29     tout[v] = T++;
30 }
31
32 bool anc(int v, int u) { return tin[v] <= tin[u] && tout[v] >= tout[u]; }
33
34 int lca(int v, int u) {
35     if (anc(v, u)) return v;
36     if (anc(u, v)) return u;
37     for (int i = lg - 1; i >= 0; i--) {
38         if (!anc(up[v][i], u)) {
39             v = up[v][i];
40         }
41     }
42     return up[v][0];
43 }
44
45 signed main() {
46     ios_base::sync_with_stdio(false);
47     cin.tie(nullptr);
48     int n, q;
49     cin >> n >> q;
50     for (int i = 0; i < maxs; i++) {
51         grupo[i] = L[i] = INF;
52         R[i] = -INF;
53     }
54     for (int i = 0; i < maxn; i++) {
55         id[i] = INF;
56     }
57     for (int i = 1; i < n; i++) {
58         int v, u;
59         cin >> v >> u;
60         adj[v].emplace_back(u);
61     }
62     dfs(1);
63     while (q--) {

```

```

64     int k;
65     cin >> k;
66     vector<int> a(k);
67     for (int i = 0; i < k; i++) {
68         cin >> a[i];
69         id[a[i]] = i;
70     }
71     sort(a.begin(), a.end(), [&](int i, int j) { return tin[i] < tin[j]; });
72     for (int i = 1; i < k; i++) {
73         a.emplace_back(lca(a[i], a[i - 1]));
74     }
75     sort(a.begin(), a.end(), [&](int i, int j) { return tin[i] < tin[j]; });
76     a.erase(unique(a.begin(), a.end()), a.end());
77
78     vector<int> stk = {a[0]};
79     for (int i = 1; i < int(a.size()); i++) {
80         int v = a[i];
81         while (!anc(stk.back(), v)) {
82             assert(int(stk.size()) >= 2);
83             int p = stk[int(stk.size()) - 2];
84             int u = stk.back();
85             f[p].emplace_back(u);
86             stk.pop_back();
87         }
88         stk.emplace_back(v);
89     }
90
91     while (int(stk.size()) >= 2) {
92         int p = stk[int(stk.size()) - 2];
93         int u = stk.back();
94         f[p].emplace_back(u);
95         stk.pop_back();
96     }
97
98     int cnt = 1;
99     int cur = 0;
100
101     auto upd = [&](int v) {
102         int x = id[v];
103         if (x == INF) return;
104         mask[x] = 1;
105         if ((x == 0 || !mask[x - 1]) && (x == k - 1 || !mask[x + 1])) {
106             grupo[x] = cnt++;
107             L[grupo[x]] = x;
108             R[grupo[x]] = x;
109         }
110         if (x > 0 && mask[x - 1]) {
111             assert(grupo[x - 1] != INF);
112             grupo[x] = grupo[x - 1];
113             R[grupo[x]] = x;
114         }

```

```

115         if (x < k - 1 && mask[x + 1]) {
116             assert(grupo[x + 1] != INF);
117             if (grupo[x] != INF) {
118                 R[grupo[x]] = R[grupo[x + 1]];
119                 grupo[R[grupo[x]]] = grupo[x];
120                 grupo[x + 1] = grupo[x];
121             } else {
122                 grupo[x] = grupo[x + 1];
123                 L[grupo[x]] = x;
124             }
125         }
126         assert(grupo[x] != INF && L[grupo[x]] != INF && R[grupo[x]] != -INF)
127         ;
128         int z = grupo[x];
129         int le = x - L[z] + 1;
130         int ri = R[z] - x + 1;
131         ans[cur] += 1LL * le * ri - (v != cur);
132         acc[cur] += 1LL * le * ri - 1;
133     };
134
135     auto add = [&](auto&& self, int v) -> void {
136         upd(v);
137         for (auto u : f[v]) {
138             self(self, u);
139         }
140     };
141
142     auto rem = [&](auto&& self, int v) -> void {
143         for (auto u : f[v]) {
144             self(self, u);
145         }
146         int x = id[v];
147         ans[v] = 0;
148         acc[v] = 0;
149         if (x < INF) {
150             mask[x] = 0;
151             grupo[x] = INF;
152         }
153     };
154
155     auto size = [&](auto&& self, int v) -> void {
156         sz[v] = 1;
157         for (auto u : f[v]) {
158             self(self, u);
159             sz[v] += sz[u];
160         }
161     };
162
163     ll ret = 0;
164     auto go = [&](auto&& self, int v, int p = -1, bool is_big = false) ->
165         void {

```

```

164         int big = -1;
165         for (auto u : f[v]) {
166             if (big == -1 || sz[big] < sz[u]) {
167                 big = u;
168             }
169         }
170         int now = cnt;
171         for (auto u : f[v])
172             if (u != big) {
173                 self(self, u, v);
174             }
175         if (big != -1) {
176             self(self, big, v, true);
177             acc[v] += acc[big];
178         }
179         cur = v;
180         for (auto u : f[v])
181             if (u != big) {
182                 add(add, u);
183             }
184         upd(v);
185         ret += 1LL * ans[v] * v;
186         if (!is_big) {
187             if (p != -1) {
188                 ans[p] -= acc[v];
189             }
190             rem(rem, v);
191             for (int i = now; i < cnt; i++) {
192                 L[i] = INF;
193                 R[i] = -INF;
194             }
195         }
196     };
197     size(size, a[0]);
198     go(go, a[0]);
199     cout << ret << endl;
200     for (auto v : a) {
201         f[v].clear();
202         sz[v] = 0;
203         id[v] = INF;
204         ans[v] = 0;
205         acc[v] = 0;
206     }
207 }
208 }

```

Problem N. Shield Navigation

In this problem, the objective is to determine whether an Almeiding can move from a cell (x_i, y_i) to another cell (x_f, y_f) in a grid of dimensions $N \times M$, considering that movements are allowed only on protected cells. A Protective Shield, centered at (x, y) , protects all cells in row x and all cells in column y . Thus, a cell (i, j) is protected if there is at least one shield built in row i or column j .

We can observe that if a cell is protected, the entire corresponding row or column is also protected, which ensures that any two protected cells are connected by a safe path. Therefore, to answer a type-2 query, it suffices to check whether both the origin and destination cells are protected. If both are protected, there is a safe path between them; otherwise, the movement is not possible.

Complexity: $O(Q)$.

Listing 14: N.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define endl '\n'
4
5  typedef long long ll;
6
7  int main() {
8      ios_base::sync_with_stdio(0);
9      cin.tie(0);
10
11     int n, m, q;
12     cin >> n >> m >> q;
13     vector<int> a(n), b(m);
14     while (q--) {
15         int id;
16         cin >> id;
17         if (id == 1) {
18             int i, j;
19             cin >> i >> j;
20             i--;
21             j--;
22             a[i] = 1;
23             b[j] = 1;
24         } else {
25             int i1, j1;
26             cin >> i1 >> j1;
27             i1--;
28             j1--;
29             int i2, j2;
30             cin >> i2 >> j2;
31             i2--;
32             j2--;
33             cout << (((a[i1] || b[j1]) && (a[i2] || b[j2]))) ? "SIM" : "NAO") <<
                 endl;
34         }
35     }
36 }
```

Problem O. Osmos V

In this problem, Malu knows that the total journey lasts X years and that Y years have already passed. Therefore, the number of years remaining to reach the destination is given by $X - Y$.

Complexity: $O(1)$.

Listing 15: O.cpp

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3
4 using namespace std;
5
6 signed main() {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9     int x, y;
10    cin >> x >> y;
11    cout << x - y << endl;
12 }
```